

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 8

Operating Systems

The Same Calls, Inside Your Own OS

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Environment 3

2.2 Part 1 - run inside xv6 3

2.3 Part 2 - pingpong 5

2.4 Optional - Primes 6

3 Conclusions **7**

A Submitted source code **8**

A.1 user/run.c 8

A.2 user/runls.c 8

A.3 user/runmissing.c 9

A.4 user/pingpong.c 10

A.5 user/primes.c 11

1 Introduction

Repeat the Linux process lifecycle in xv6, implement a two-pipe byte exchange, and extend the pipe mechanism into a process-chain prime sieve.

The experiments follow the laboratory handout and were performed inside Ubuntu 26.04.1 LTS in VirtualBox. The figures show actual terminal screenshots taken during execution; complete transcripts and the submitted source files accompany the report.

2 Tasks and results

2.1 Environment

All experiments were performed on 23 October 2026 inside **Ubuntu 26.04.1 LTS (ARM64)** running in VirtualBox as user `misa` on `UbuntuVM`. The Linux kernel is `7.0.0-38-generic`. The installed xv6 checkout is `~/xv6-riscv`, commit `06aad25c735fd3159bdfae5680be4eab7b1668b2`. GCC and the RISC-V compiler are version 15.2.0; QEMU is version 10.2.1. xv6 is booted from this Ubuntu with `make qemu CPUS=1`.

The figures are screenshots of the Ubuntu terminal during execution. Complete command output is provided in `terminal-output.txt`; build logs are in `build-output.txt` for the xv6 labs. PIDs, inode numbers and free-space values below refer to these recorded runs.

The xv6 image was rebuilt for this laboratory. Its Makefile retains the earlier lab utilities and adds run, runls, runmissing, pingpong and primes.

```
misa@UbuntuVM:xv6-riscv$ grep -n "_run\|_pingpong\|_primes" Makefile
157:    $U/_run\
158:    $U/_runls\
159:    $U/_runmissing\
160:    $U/_pingpong\
161:    $U/_primes\
misa@UbuntuVM:xv6-riscv$ grep nmeta /home/misa/os-labs-04-08-20261023/evidence/lab08-build.txt
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
misa@UbuntuVM:xv6-riscv$
```

Figure 2.1 - The required programs are registered in UPROGS and the xv6 image builds inside Ubuntu.

2.2 Part 1 - run inside xv6

`user/run.c` follows the provided program. The parent prints its PID and forks; the child prints its own PID and calls exec with a NUL-terminated argument vector for echo. The parent waits for the child and prints the returned status. The ls variant is `user/runls.c`; the failed-exec variant is `user/runmissing.c`, leaving run as the default echo example.

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$ runls
parent: my pid is 5
child: my pid is 6
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
sleep          2 23 35280
args           2 24 35272
mystat         2 25 36456
find           2 26 44496
run            2 27 35888
runls          2 28 35888
runmissing     2 29 35896
pingpong       2 30 37176
primes         2 31 42048
console        3 32 0
parent: child 6 ended with status 0
$ runmissing
parent: my pid is 7
child: my pid is 8
exec failed
parent: child 8 ended with status 1
$
```

Figure 2.2 - Echo, ls and failed-exec variants executed in the xv6 booted from Ubuntu.

The first run used parent PID **3** and child PID **4**, printed `hello from the child`, and reported status 0. runls used PIDs 5/6, displayed the xv6 root listing and also reported 0. The nonexistent executable used PIDs 7/8, printed `exec failed` and reported status 1.

2.2.1 1. Compare the three calls with Linux

Operation	**xv6 lab 08**	**Linux lab 07**
Create the child	`fork()`	`fork()`
Replace its image	`exec("echo", args)`	`execlp("ls", "ls", "-l", (char *)NULL)`
Reap it	`wait(&status)`	`waitpid(pid, &status, 0)`

The process lifecycle is the same: fork creates a child with inherited memory contents and descriptors; exec replaces its program while retaining its PID; wait collects its termination information and reaps it. Linux execlp is a libc helper with PATH search and variadic arguments. xv6 exec takes a pathname and an argument vector directly, without PATH search. xv6 wait stores the exit value directly, whereas Linux waitpid returns an encoded status requiring wait macros.

2.2.2 2. Who prints hello from the child?

The echo executable prints it. run prints the PID messages before exec; successful exec never returns to that old image. echo receives the words in its argument vector and writes the resulting message.

2.2.3 3. Why failure is 1 here and 127 on Linux

The xv6 branch explicitly calls `exit(1)` after exec fails, while the Linux branch explicitly calls `exit(127)`. The programs choose those numbers; exec returns an error rather than automatically assigning an exit code. The xv6 parent still exits 0 after reporting the child, following the supplied example.

2.3 Part 2 - pingpong

`user/pingpong.c` creates parent-to-child and child-to-parent pipes and forks. Each process closes the ends it does not use. The parent sends one initialized byte; the child reads it, prints ping, sends the same byte back and exits. The parent reads the reply, prints pong, closes the remaining ends and reaps the child. Failures from pipe, fork, read, write and wait are checked.

```
parent p2c[1] -- one byte --> p2c[0] child
parent c2p[0] <-- one byte -- c2p[1] child
```

```
$ pingpong
10: received ping
9: received pong
$
```

Figure 2.3 - The child prints received ping before its parent prints received pong.

The child PID was **10** and the parent PID was **9**. The child prints ping before writing the reply, and the parent prints pong only after receiving that byte. This data dependency guarantees the message order even if scheduling changes the timing.

2.3.1 1. Why two pipes rather than one

A pipe is a one-way byte stream with no addressed messages. A separate pipe for each direction gives each process a distinct input and output channel. If both processes read and write the same pipe, either could consume its own byte instead of the intended peer receiving it.

2.3.2 2. Which call makes the parent wait for the reply

`read(c2p[0], &b, 1)` blocks until the child supplies a byte while a writer remains open. The later wait call reaps the child; it is not what synchronizes reception of the reply. Closing unused writers also makes EOF possible when the peer exits without replying.

2.4 Optional - Primes

`user/primes.c` generates integers 2 through 35 and creates a chain of filtering processes. Each filter reads its first number as a prime, prints it, and forwards only numbers not divisible by it to the next stage. Each process closes its unused pipe ends and its output end when input reaches EOF, then waits for its child.

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
$
```

Figure 2.4 - The process-chain sieve prints every prime from 2 through 35 and terminates.

The actual output is **2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31**, exactly the eleven primes in that inclusive interval. Closing the final writers propagates EOF through the chain and lets every filter exit. Retaining only necessary descriptors also respects xv6's per-process descriptor limit.

3 Conclusions

The xv6 process lifecycle follows the same fork-exec-wait pattern as Linux, with smaller interfaces and explicit executable paths. The failure status comes from the exit call chosen by the program. The two-pipe exchange guarantees ping before pong because the parent blocks for the reply. The optional sieve demonstrates how correct descriptor closure propagates EOF through a chain of processes.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 8: *The Same Calls, Inside Your Own OS*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, commit `06aad25c735fd3159bdfae5680be4eab7b1668b2`. Source checked on 23 October 2026. `https://github.com/mit-pdos/xv6-riscv/tree/06aad25c735fd3159bdfae5680be4eab7b1668b2`.

[3] Linux man-pages project. *fork(2), pipe(7), inode(7), path_resolution(7)*. `https://man7.org/linux/man-pages/`.

A Submitted source code

The following listings read the actual submitted source files, so the report and submission use the same implementation. Variant programs are included where relevant.

A.1 user/run.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

A.2 user/runls.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "ls", 0 };
    exec("ls", args);
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

A.3 user/runmissing.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("nosuchprog", args);
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

A.4 user/pingpong.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];
  char b = 'p';
  if(pipe(p2c) < 0){
    fprintf(2, "pingpong: pipe failed\n"); exit(1);
  }
  if(pipe(c2p) < 0){
    close(p2c[0]); close(p2c[1]);
    fprintf(2, "pingpong: pipe failed\n"); exit(1);
  }
  int pid = fork();
  if(pid < 0){
    close(p2c[0]); close(p2c[1]); close(c2p[0]); close(c2p[1]);
    fprintf(2, "pingpong: fork failed\n"); exit(1);
  }
  if(pid == 0){
    close(p2c[1]); close(c2p[0]);
    if(read(p2c[0], &b, 1) != 1){
      fprintf(2, "pingpong: read ping failed\n"); exit(1);
    }
    close(p2c[0]);
    printf("%d: received ping\n", getpid());
    if(write(c2p[1], &b, 1) != 1){
      fprintf(2, "pingpong: write pong failed\n"); exit(1);
    }
    close(c2p[1]);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);
  int failed = write(p2c[1], &b, 1) != 1;
  close(p2c[1]);
  if(!failed && read(c2p[0], &b, 1) == 1)
    printf("%d: received pong\n", getpid());
  else {
    fprintf(2, "pingpong: exchange failed\n"); failed = 1;
  }
  close(c2p[0]);
  int status;
  if(wait(&status) < 0 || status != 0)
    failed = 1;
  exit(failed);
}
```

A.5 user/primes.c

```c
#include "kernel/types.h"
#include "user/user.h"

/* One filter per prime. Mark noreturn so recursive process creation is clear. */
static void filter(int input) __attribute__((noreturn));

static void
filter(int input)
{
  int prime, value, p[2];
  int n = read(input, &prime, sizeof(prime));
  if(n == 0){ close(input); exit(0); }
  if(n != sizeof(prime)){ fprintf(2, "primes: read failed\n"); exit(1); }
  printf("prime %d\n", prime);
  if(pipe(p) < 0){ fprintf(2, "primes: pipe failed\n"); exit(1); }
  int pid = fork();
  if(pid < 0){ fprintf(2, "primes: fork failed\n"); exit(1); }
  if(pid == 0){ close(input); close(p[1]); filter(p[0]); }
  close(p[0]);
  while((n = read(input, &value, sizeof(value))) == sizeof(value)){
    if(value % prime && write(p[1], &value, sizeof(value)) != sizeof(value)){
      fprintf(2, "primes: write failed\n"); exit(1);
    }
  }
  close(input); close(p[1]);
  int status;
  if(wait(&status) < 0 || status != 0 || n != 0)
    exit(1);
  exit(0);
}

int
main(int argc, char *argv[])
{
  int p[2];
  if(pipe(p) < 0){ fprintf(2, "primes: pipe failed\n"); exit(1); }
  int pid = fork();
  if(pid < 0){ fprintf(2, "primes: fork failed\n"); exit(1); }
  if(pid == 0){ close(p[1]); filter(p[0]); }
  close(p[0]);
  for(int value = 2; value <= 35; value++)
    if(write(p[1], &value, sizeof(value)) != sizeof(value)){
      fprintf(2, "primes: write failed\n"); exit(1);
    }
  close(p[1]);
  int status;
  if(wait(&status) < 0 || status != 0)
    exit(1);
  exit(0);
}
```